

SO2 – Etapa 3

Sincronização Temporal

Integrantes: Arthur Erpen, Caetano Peruzzo e Gabriel Herrera

UFSC - Sistemas Operacionais II

Maio de 2026

Problema da Etapa

- A partir de agora, as mensagens precisam carregar um *timestamp* de origem
- As VMs precisam sincronizar os seus relógios com precisão suficiente, que possibilite a reconstrução da ordem de produção das mensagens
- Entre os componentes de uma mesma VM a percepção de tempo é compartilhada

Decisões de implementação

- Existe uma RSU que é a responsável por prover a base para sincronização temporal das VMs
- Os gateways são os agentes do SPTP
- Dentro de cada VM, o componente *gateway* é responsável por sincronizar o relógio *CLOCK_REALTIME*, o que afeta a percepção de tempo de todos os componentes da VM
- A RSU tem um *gateway* que é o agente do SPTP

Protocolo SPTP – Papeis

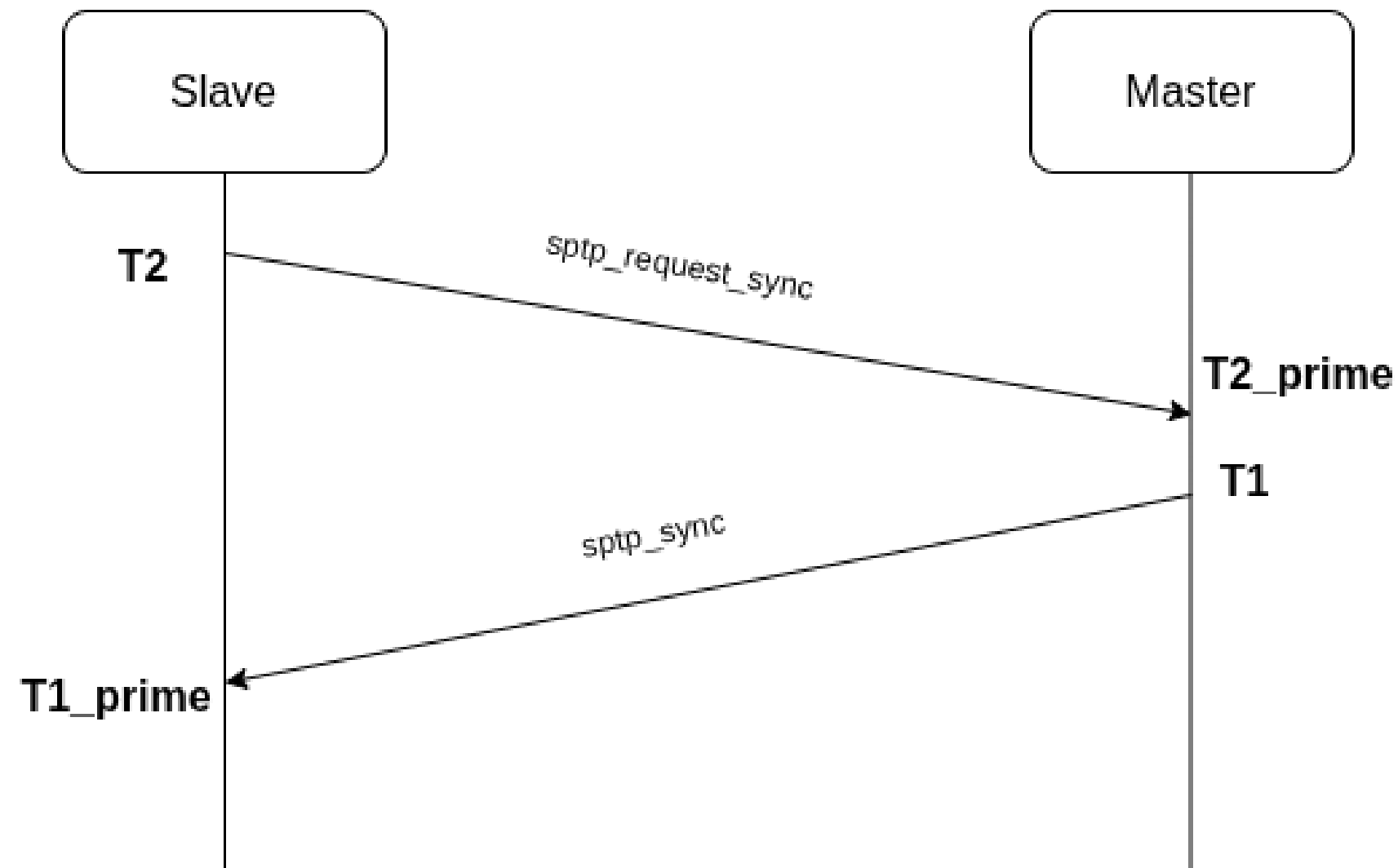
Master

- Possui a percepção de tempo base, que funciona como alicerce para o funcionamento do SPTP
- Recebe mensagens SPTP marcadas como **sptp_request_sync**, e responde com uma mensagem **sptp_sync**, em unicast

Slave

- Responsável por enviar mensagens SPTP **sptp_request_sync**, em broadcast, iniciando o processo de sincronização do próprio relógio
- Recebe mensagens SPTP marcadas como **sptp_sync**, e coleta os dados enviados

Protocolo SPTP

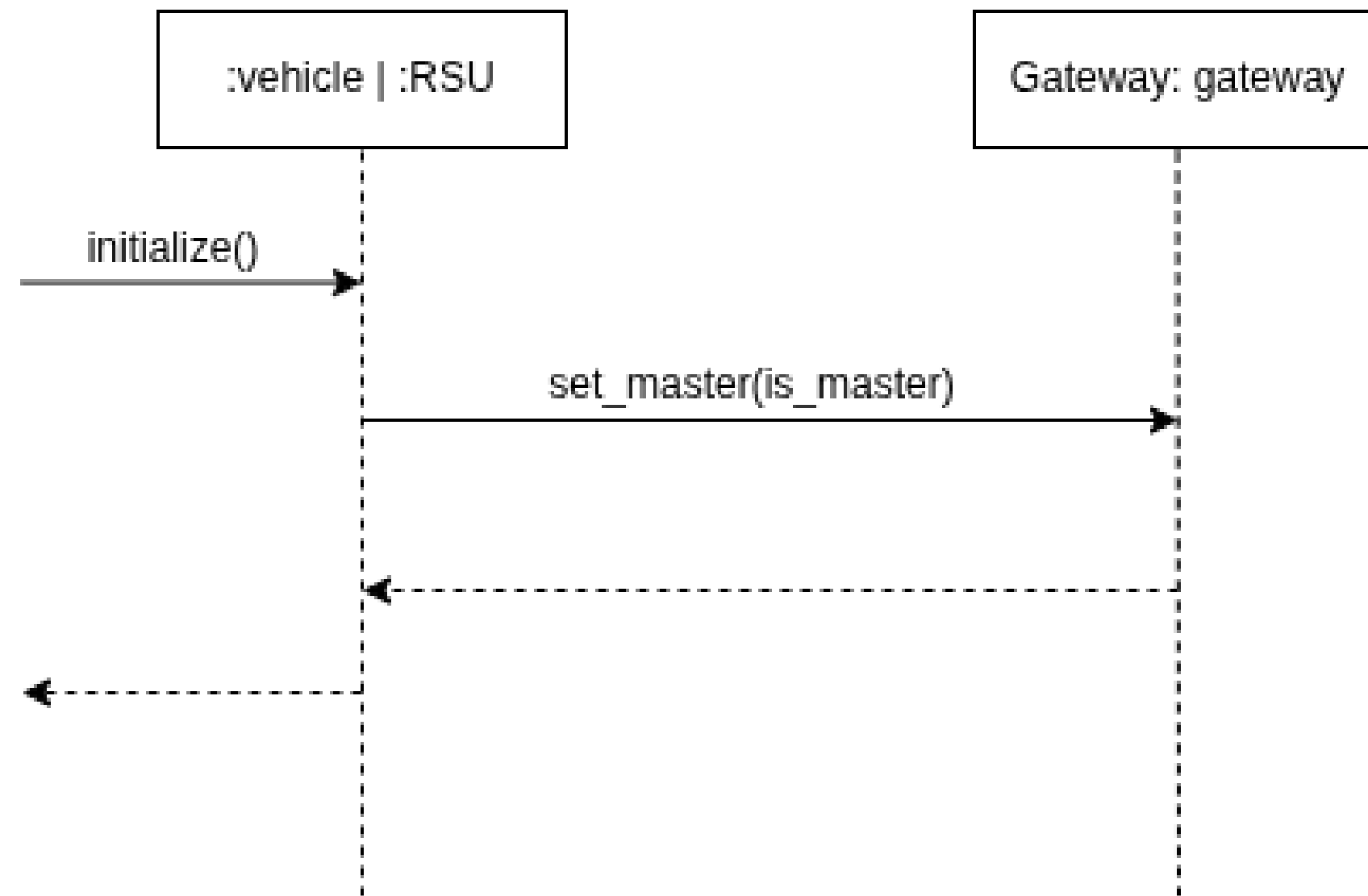


$$offset = \frac{(t_{1prime} - t_1) - (t_{2prime} - t_2)}{2}$$

$$delay = \frac{(t_{1prime} - t_1) + (t_{2prime} - t_2)}{2}$$

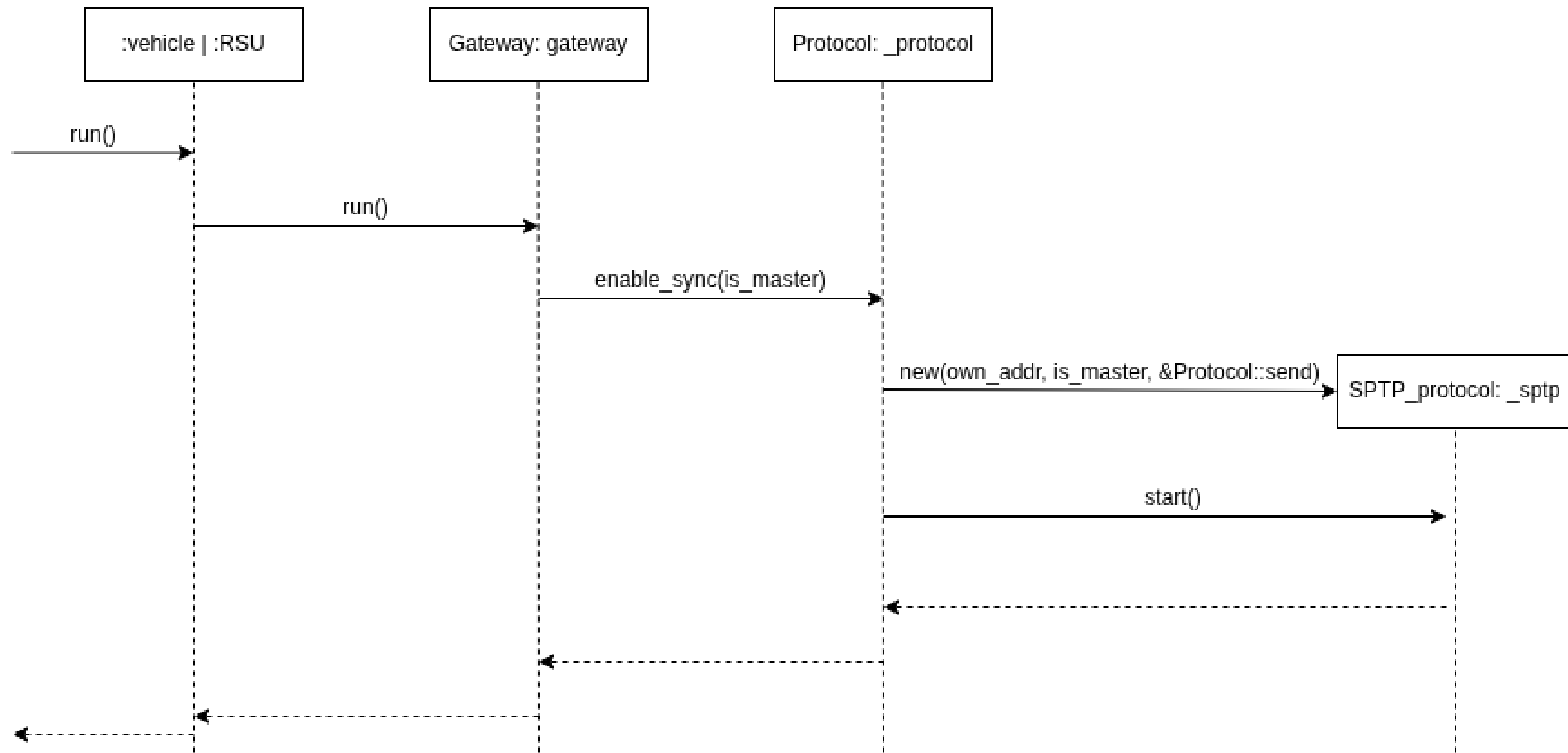
- O SPTP é *slave driven*, ou seja, o *slave* manda a mensagem, o *master* apenas responde
- O *slave* também é responsável por coletar cada um dos *timestamps* da comunicação, assim como calcular o *offset* e o *delay*

Protocolo SPTP – Inicialização



- A inicialização do veículo e da RSU é responsável por definir o papel da VM no SPTP
- A RSU sempre é master

Protocolo SPTP – Inicialização

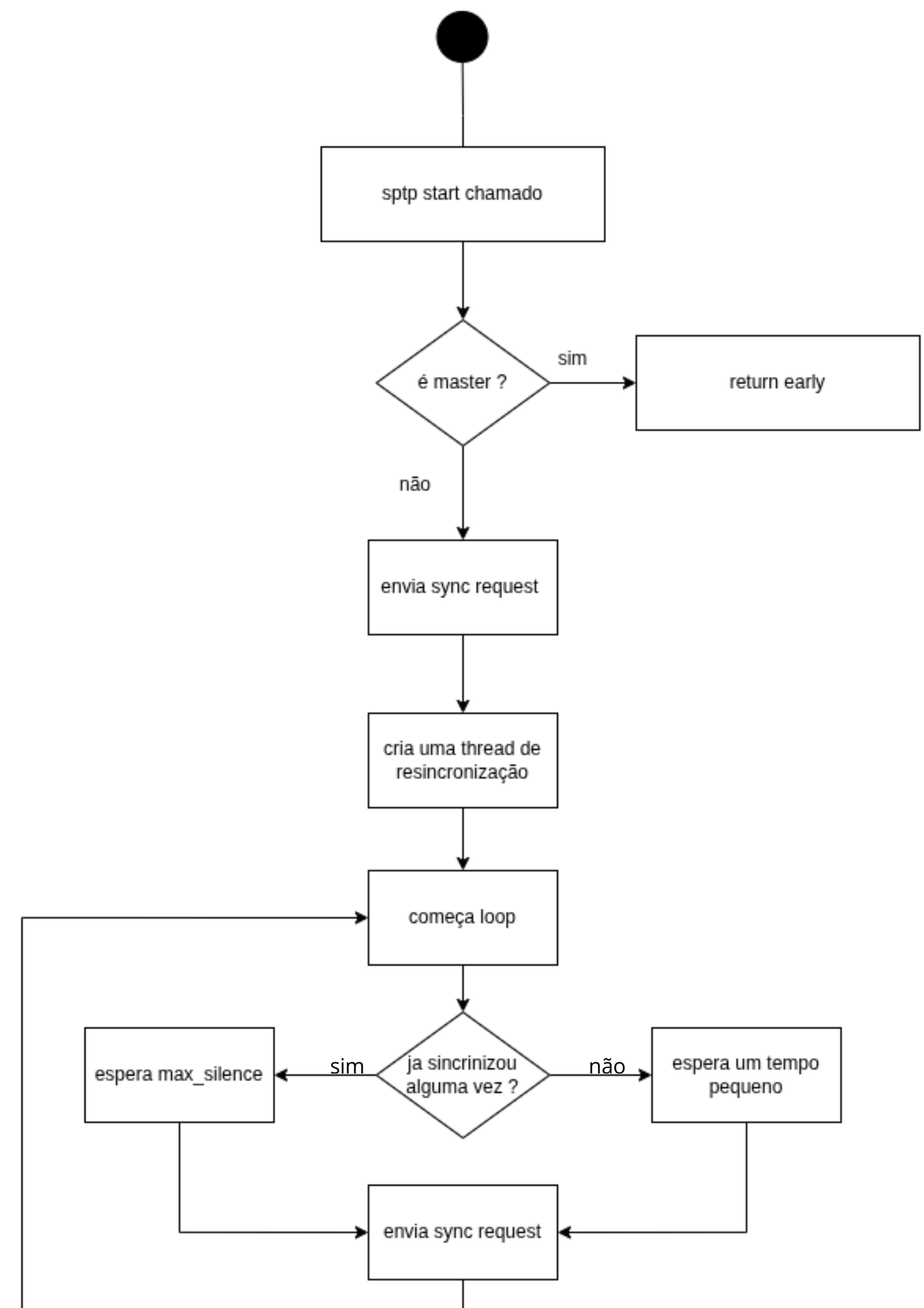


- Cria uma instância do objeto SPTP_Protocol, com um ponteiro para o método send do *protocol* da pilha, o que permite autonomia do SPTP no envio de mensagens.

Protocolo SPTP – Sincronização

Os agentes slave enviam periodicamente solicitações de sincronização com o objetivo de manter seus relógios alinhados ao relógio de referência com boa precisão.

O intervalo entre essas mensagens de resincronização deve ser determinado experimentalmente, de forma que o offset entre os relógios das máquinas virtuais nunca ultrapasse um limite máximo previamente estabelecido.



Protocolo SPTP – Sincronização

O *drift* médio foi estimado a partir da média dos valores de *drift* observados ao longo da execução de 60s do cenário *sptp-drift*. o *drift* médio é usado para estipular o tempo de meia vida da validade de um relógio, considerando o erro máximo como 1ms .

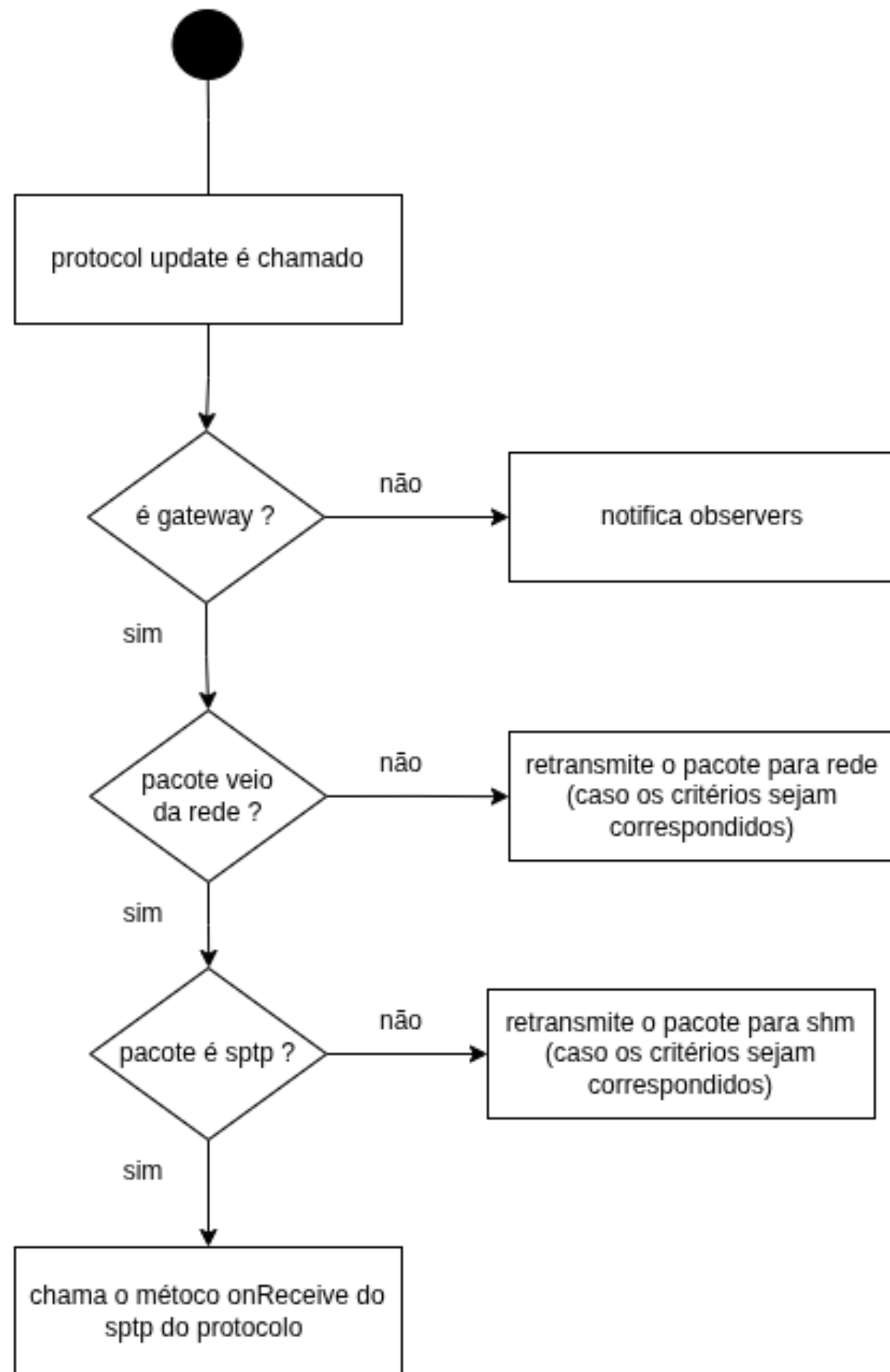
$$slope = \frac{drift}{\Delta t} \quad meia_vida = \frac{err_{max}}{2 * slope} \quad err_{max} = 1000\mu s$$

considerando um valor que obtemos em testes de drift médio absoluto de = 101us

$$slope = 1.68\mu s/s \quad meia_vida = 297.62s$$

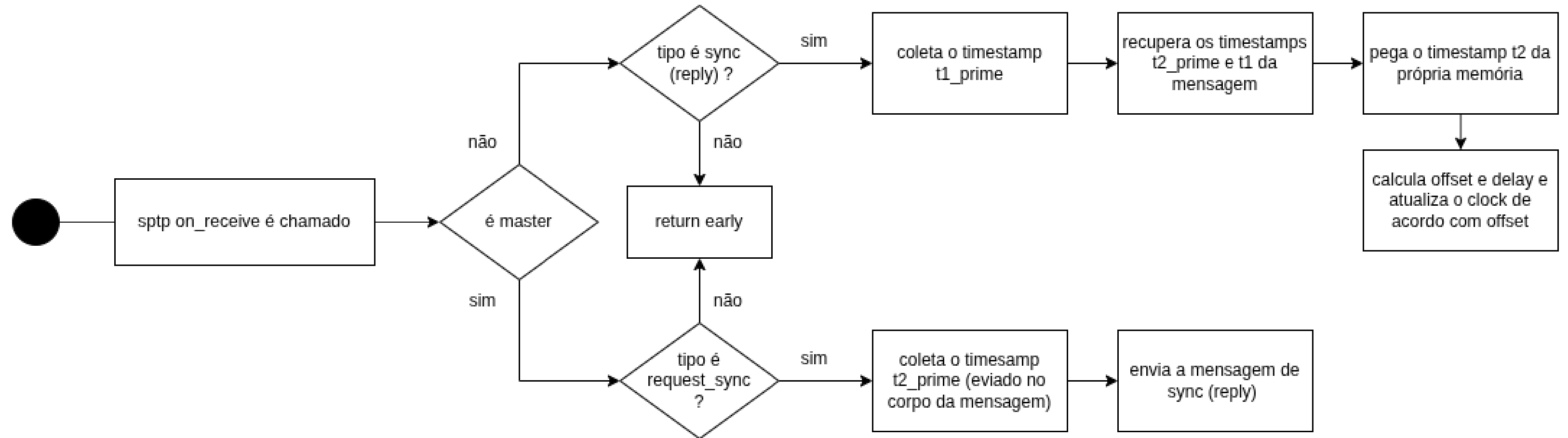
Por tanto percebemos que nosso valor padrão de 15s é suficiente para manter a validade dos relógios das VMs

Protocolo SPTP – Recebimento



- o SPTP é ativado dentro do método `Protocol::update()`, isso acontece quando:
- o *protocol* é de um *gateway*
 - o pacote veio da rede
 - o tipo do pacote (*packet_kind*) é **spt_sync** ou **sptp_request_sync**

Protocolo SPTP – Recebimento



- O timestamp t1 é coletado e pelo método *protocol::send_via_nic()* e é o header de timestamp da mensagem

test-sptp-simple

- 6 VMs: 1 RSU master SPTP, 2 senders e 3 receivers.
- A RSU atua apenas como referência de tempo, respondendo às requisições de sincronização SPTP.
- Os senders transmitem mensagens periódicas em broadcast, cada uma marcada com timestamp no momento do envio.
- O segundo sender começa 250 ms depois do primeiro, criando mensagens próximas no tempo, mas com origens diferentes.
- As amostras são separadas por origem, validando que mensagens são identificadas de forma inequívoca por (origin, timestamp).
- O teste passa quando o erro médio absoluto fica abaixo do limite definido, confirmando a sincronização SPTP.

test-sptp-simple

Receiver	Sender	samples	avg_abs_us
VM4	VM2	22	2354
VM4	VM3	22	1579
VM5	VM2	22	2275
VM5	VM3	22	2695
VM6	VM2	22	2439
VM6	VM3	22	3192

avg_abs_us: média do erro absoluto em microssegundos.

test-sptp-drift

- 3 VMs: RSU master, transmissor e receiver de medição.
- Faz uma sincronização inicial e desativa ressincronizações durante o teste.
- O transmissor envia mensagens por cerca de `60 s`.
- O receiver mede quanto o relógio deriva em relação a baseline inicial.

Uso:

- observar drift sem nova sync
- justificar o intervalo de ressincronizacao do SPTP

measure-rtt

Metrica	Valor
RTT medio	8.2455 ms
RTT minimo	2.7920 ms
RTT maximo	17.2960 ms
Latencia estimada, RTT/2	4.1228 ms

measure-rtt-intra

Metrica	Valor
RTT medio	0.9212 ms
RTT minimo	0.8190 ms
RTT maximo	9.8560 ms
Latencia estimada, RTT/2	0.4606 ms

Correções etapa 2

Mensagem com MAC repetido

- Antes, o endereço MAC aparecia duplicado em uma mesma mensagem: uma vez no cabeçalho da própria mensagem e outra no cabeçalho do frame Ethernet.
- Agora, o MAC presente no frame Ethernet é apenas copiado para um campo da mensagem. mas, esse campo não faz parte do cabeçalho da mensagem em si, servindo apenas como uma forma de tornar essa informação acessível às camadas superiores.

Contador de Readers left na SHM

- O professor comentou que tínhamos um vetor de semáforos pending/full (um para cada componente) e um contador de readers_left para cada slot da SHM, e que talvez uma dessas estruturas fosse desnecessária.
- Agora não existe mais o contador de readers_left, a cada leitura, o leitor consegue determinar se é o último a consumir aquele slot apenas pela posição dos cabeçotes de leitura.

Obrigado!